

Systematic Benchmarking of Cooperative Multi-Agent Reinforcement Learning Algorithms in VMAS Using BenchMARL

Andrea Stevanoska Viktor Kostadinovski Gorazd Filipovski
Sonja Gievska Martina Toshevska

Faculty of Computer Science and Engineering

February 2026

Abstract

This project investigates cooperative Multi-Agent Reinforcement Learning in simulated continuous-control environments. We adopt BenchMARL, a benchmarking and training framework that standardizes MARL experimentation through a unified configuration interface separating algorithm settings, model architectures, global training parameters, and task-specific configurations, enabling reproducible comparisons. Experiments are conducted in VMAS (Vectorized Multi-Agent Simulator) on four structured cooperative tasks: Reverse Transport, Joint Passage Size, Multi Give Way and Balance. We evaluate representative CTDE and value-decomposition methods-MADDPG, MASAC, MAPPO, QMIX, and VDN using a systematic benchmarking workflow with baseline runs followed by iterative hyperparameter refinement and centralized tracking in Weights & Biases. Results indicate that algorithm suitability depends strongly on task structure: MAPPO is most effective in the constrained coordination setting of Joint Passage Size and Balance, while MASAC and MADDPG exhibit stronger performance in interaction-heavy give-way tasks; Reverse Transport is solved reliably after tuning informed by earlier experiments.

1 Introduction

Cooperative multi-agent systems require agents to coordinate under shared objectives, partial observability, and dynamic interaction constraints. Multi-Agent Reinforcement Learning (MARL) provides a general framework for learning such behaviors directly from experience. However, practical training remains challenging due to non-stationarity, credit assignment difficulties, reward shaping sensitivity, and the presence of locally stable but suboptimal coordination strategies. These challenges are amplified in continuous-control environments with coupled physical dynamics, where small policy changes can significantly alter collective behavior.

In recent years, centralized training with decentralized execution (CTDE) methods such as MAPPO and MADDPG have demonstrated strong empirical performance in cooperative settings. At the same time, value-decomposition methods such as VDN and QMIX have shown success in discrete-action domains. Despite these advances, systematic comparisons across continuous-control tasks remain limited, and performance is often highly sensitive to task configuration and reward design. This makes reproducible benchmarking and structured experimentation essential.

BenchMARL provides a standardized framework for MARL experimentation, separating algorithm, model, experiment, and task configurations into modular components. Using this framework together with the VMAS continuous multi-agent simulator, we conduct a systematic empirical study across multiple cooperative tasks exhibiting distinct coordination patterns: constrained passage (Joint Passage Size), right-of-way negotiation (Give Way and Multi Give Way), and synchronized object transport (Reverse Transport and Balance).

Rather than proposing a new algorithm, this work focuses on understanding algorithm suitability under different coordination structures. We begin with controlled baseline evaluations across MAPPO, MADDPG, MASAC, QMIX, and VDN, followed by iterative hyperparameter refinement and reward shaping analysis. Our results show that (i) algorithm performance depends strongly on task structure and (ii) reward design can induce competing behavioral regimes within the same task. In particular, the Joint Passage Size task reveals a multi-objective tension between structured parallel traversal and terminal parking precision.

Through detailed quantitative analysis and qualitative behavioral inspection, this study provides practical insights into the interaction between algorithm design, reward shaping, and environment structure in continuous cooperative MARL systems.

The remainder of this paper is organized as follows. Section 2 reviews related work in cooperative MARL and benchmarking frameworks. Section 3 describes the experimental framework and methodology, including the BenchMARL configuration structure, evaluated algorithms, and training protocol. Section 4 presents the task-specific experimental results and comparative analyses across environments. Section 5 discusses limitations and outlines directions for future work. Finally, Section 6 concludes the paper with key findings and overarching insights.

2 Related Work

Multi-Agent Reinforcement Learning (MARL) has a long research history motivated by the need for coordinated decision-making in domains such as distributed control, robotics, and resource management. Early foundational work emphasized that multi-agent learning introduces challenges absent in single-agent RL, particularly the non-stationarity induced by concurrently learning agents and the difficulty of defining meaningful multi-agent learning goals. Busoniu et al. [3] provide a comprehensive overview of MARL and highlight two central objectives that recur throughout the literature: stabilizing learning dynamics and adapting to other agents’ changing behaviors. These issues

are directly relevant to cooperative continuous-control tasks, where coupled dynamics and shared rewards can yield locally stable yet suboptimal coordination patterns.

More recent surveys further systematize MARL theory and practice and emphasize how different modeling assumptions lead to different algorithm families. Zhang et al. [5] present a selective overview focused on theoretically grounded MARL, organizing methods through frameworks such as Markov/stochastic games and extensive-form games, and discussing (non-)convergence properties of policy-based learning in games. This perspective helps contextualize our observations that policy-gradient CTDE methods can learn stable cooperation in some tasks, yet may also converge to locally optimal behavioral regimes under scalarized rewards. Complementarily, Canese et al. [4] provide a practical taxonomy of widely used MARL approaches and compare them along key axes-non-stationarity, scalability, and observability-while also discussing commonly used benchmark environments for empirical evaluation.

Despite this substantial body of work, empirical performance is often difficult to compare across papers due to differences in environment implementations, reward shaping, and tuning protocols. This motivates standardized benchmarking pipelines that enable controlled experimentation and reproducibility. Our work follows this direction by using BenchMARL [2] and the VMAS simulator [1] to evaluate representative cooperative MARL algorithms under consistent training and evaluation procedures. In particular, our experiments examine how algorithm design choices and task configuration interact in continuous-control environments, and we report both quantitative learning curves and qualitative emergent behaviors across coordination tasks.

3 Experimental Framework and Methodology

The experiments were conducted using the BenchMARL framework, which provides a modular and configurable environment for multi-agent reinforcement learning research. The framework allows explicit selection of the learning algorithm, neural network architecture, task environment, and experiment-level parameters. This structured separation ensures controlled comparisons and reproducibility across experimental runs.

Four configuration layers are defined within the framework:

- **Algorithm configuration:** Determines the learning algorithm (e.g., MADDPG, MASAC, MAPPO) and specifies its internal properties such as critic structure, entropy regularization, loss formulation, and target update rules.
- **Model configuration:** Specifies the neural network architecture used by the agents. Available options include CNN, GRU, GNN, LSTM, and MLP models. The Multi-Layer Perceptron (MLP) was selected as a consistent baseline architecture to focus the analysis on algorithmic and task-level variations.
- **Experiment configuration:** Defines global training parameters including learning rate, discount factor, batch sizes, number of iterations, evaluation intervals, replay buffer settings (for off-policy methods), and checkpoint scheduling. A

single experiment-level baseline configuration was maintained to ensure fairness across algorithms, with only targeted adjustments introduced during hyperparameter refinement.

- **Task configuration:** Selects the VMAS task environment and defines environment-specific parameters such as reward shaping, physical properties, episode length, and observation settings.

In addition to using the MLP as the baseline architecture, limited exploratory runs were conducted with alternative model configurations. These variations did not produce meaningful qualitative or quantitative improvements across tasks. As a result, further experiments focused primarily on algorithm-level and task-level configurations rather than architectural changes.

Training is carried out using multiple parallel environments to collect experience. Off-policy methods store this experience in replay buffers and update the networks using sampled batches. Target networks are updated regularly depending on the algorithm. Evaluation, logging, and checkpointing are performed throughout training to track performance and ensure reproducibility.

3.1 Selected Tasks

Five cooperative tasks from the VMAS suite were trained and evaluated. Four primary tasks were selected to represent distinct coordination challenges, and one simpler variant was included to study reduced-complexity behavior.

Reverse Transport. Agents cooperatively move a shared object to a target location. The object’s mass requires synchronized force application and coordinated movement.

Joint Passage Size. Agents must coordinate their positions to pass through a constrained opening without causing blockage.

Multi Give Way. Multiple agents approach a shared corridor from opposing directions and must negotiate right-of-way through implicit role assignment.

Give Way. A simplified two-agent version of Multi Give Way in which one agent must move forward while the other waits to avoid blocking.

Balance. Agents cooperatively stabilize and move a shared object to an end goal under continuous physical dynamics.

3.2 Evaluated Algorithms

Five cooperative multi-agent reinforcement learning algorithms were evaluated:

- MADDPG
- MASAC
- MAPPO
- QMIX (CMIX)
- VDN

Additional algorithms available within BenchMARL include ISAC, IQL, IPPO, and IDDPG. These were not evaluated in this study, as they are generally more suited to independent or competitive multi-agent settings. The selected algorithms were chosen specifically for cooperative environments, either through centralized critic mechanisms or value decomposition strategies.

QMIX and VDN were tested in several runs but showed weak performance and unstable coordination on the selected continuous-control tasks. These algorithms use value factorization, which is generally more suitable for discrete or simpler cooperative problems. The strong interaction between agents and the more complex coordination required in the VMAS tasks likely made these methods less effective, so further tuning was not prioritized.

3.3 Experimental Strategy

An iterative hyperparameter exploration approach was applied.

Baseline Runs. Each algorithm was first trained using a shared baseline experiment configuration together with the default task, algorithm, and model parameters provided by BenchMARL. This baseline setup was applied consistently across all selected tasks and algorithms in order to establish a controlled reference point for stability and performance comparison.

Algorithm Selection. Algorithms demonstrating persistent instability or weak cooperative behavior were deprioritized, while promising methods were selected for further tuning.

Hyperparameter Refinement. Learning rates, reward shaping parameters, target update rates, exploration settings, and episode horizons were systematically adjusted. Approximately twenty runs per task were conducted across different random seeds and configurations.

Logging and Reproducibility. All experiments were tracked using Weights & Biases (WandB). Evaluation metrics, training curves, and model checkpoints were recorded at regular intervals to ensure reproducibility and fair comparison.¹

¹WandB project dashboards: Joint Passage Size — <https://wandb.ai/itonkdong-org/benchmarl?nw=nwuseritonkdong>; Give Way, Multi Give Way, and Reverse Transport —

4 Task-Specific Experimental Results

4.1 Joint Passage Size

Task Description and Coordination Requirements The *Joint Passage Size* task is a cooperative continuous-control problem within the VMAS framework, involving two physically connected agents that must navigate through spatial constraints and reach designated goal positions. Due to their mechanical coupling, the agents’ actions directly influence one another, requiring coordinated motion at every time step. Successful completion demands synchronized control of position and orientation under continuous dynamics, where implicit cooperation emerges through shared rewards rather than explicit communication.

The environment contains multiple gates of different sizes. While a larger gate allows passage without strict alignment, narrower gates require precise parallel coordination to avoid collisions and instability. This creates a core challenge: agents must learn structured alignment behavior rather than relying on independent or greedy movements, which typically result in oscillations or deadlocks.

A further complexity arises from the presence of multiple feasible strategies. Although parallel traversal through constrained gates represents the intended cooperative solution, agents may instead exploit the larger gate, achieving task completion without developing fine-grained coordination. The task therefore evaluates not only goal-reaching ability but the emergence of stable, collision-minimized cooperative behavior under coupled continuous dynamics.

4.2 Baseline Performance Across Algorithms

All five MARL algorithms (MAPPO, MADDPG, MASAC, VDN, and QMIX) were initially trained using the default BenchMARL configurations under identical environment and model settings. As shown in Figures 1 and 2, MAPPO achieved the highest final reward and exhibited the most consistent upward learning trend. Although moderate oscillations were present, no divergence or collapse was observed, and evaluation rewards confirmed stable generalization. MADDPG showed partial improvement but remained unstable, while MASAC exhibited persistent oscillations without clear convergence.

In contrast, VDN and QMIX failed to demonstrate meaningful learning progress. Their reward curves were highly unstable and frequently degraded, leading to behavioral collapse. Qualitative evaluation (Figure 3) confirmed these findings: VDN and QMIX agents remained largely stationary, MASAC agents drifted without coordination, MADDPG achieved limited but unstable coordination, and MAPPO produced the most structured movement toward the goal despite greedy gate selection and collision events.

<https://wandb.ai/andrea-stevanoska-finki/benchmarl?nw=nwuserandreastevanoska>; Balance — <https://wandb.ai/gorazd-filipovski-faculty-of-computer-science-and-engine/benchmarl?nw=nwusergorazdfilipovski>.

Based on both quantitative and qualitative analysis, MAPPO was selected for further refinement, with MADDPG retained for limited exploration. The weak performance of VDN and QMIX was expected, as both algorithms are primarily designed for discrete action spaces and value decomposition settings, whereas the Joint Passage Size task involves continuous control and tightly coupled dynamics. Their inclusion served as an empirical evaluation of their applicability in continuous MARL environments.

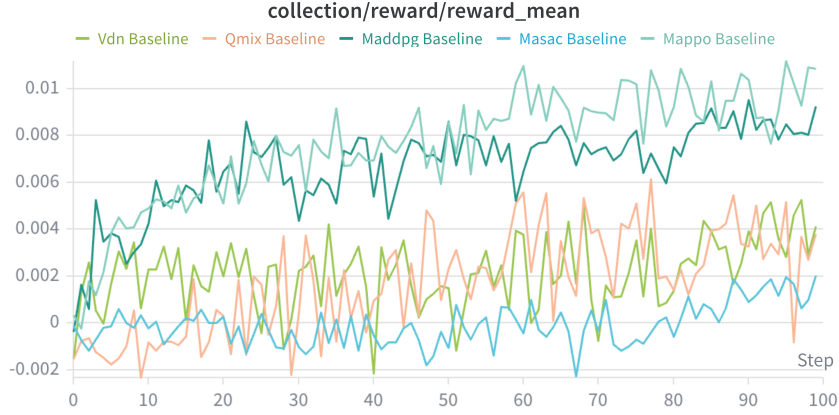


Figure 1: Mean training reward comparison across algorithms.

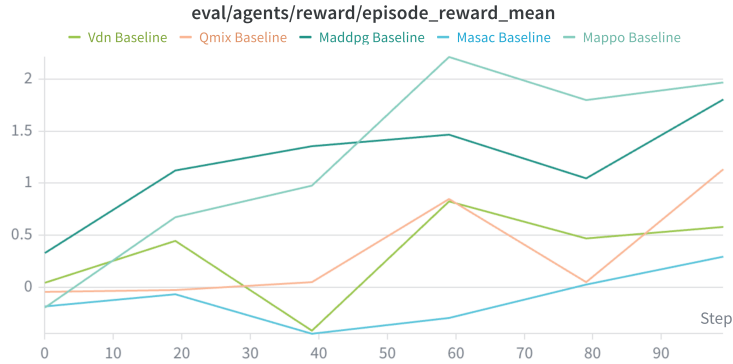


Figure 2: Evaluation episode reward comparison across algorithms.

4.3 Hyperparameter Exploration Strategy

Following the baseline evaluation, an iterative refinement process was conducted to assess the sensitivity of MAPPO to optimization and architectural parameters before modifying the task reward structure. The baseline configuration used the default

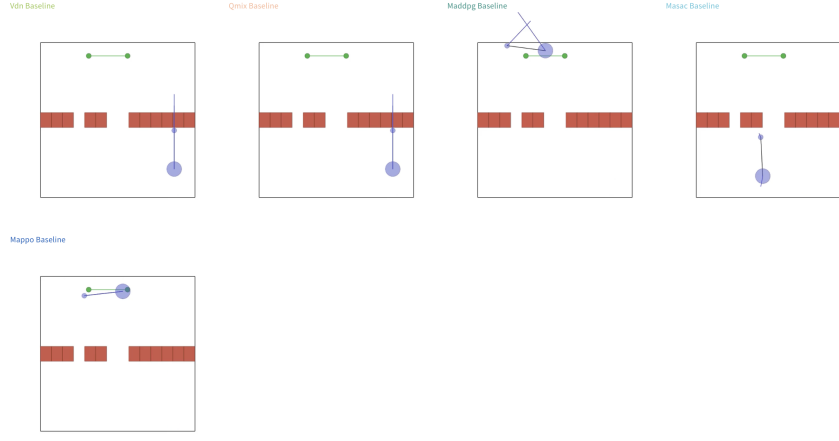


Figure 3: Final behavior comparison across algorithms.

BenchMARL settings (learning rate 5×10^{-5} , zero entropy regularization, critic coefficient 1.0, shared parameters, and a two-layer MLP with 256 units per layer). Under this setup, agents interacted with the environment but failed to achieve stable coordinated passage through constrained gates.

During baseline experimentation, multiple training horizons were evaluated to determine an appropriate number of iterations. Empirically, 100 iterations (corresponding to 600,000 environment frames in BenchMARL, with `collected_frames_per_batch` = 6000) produced the most stable learning dynamics. Extending training beyond this point frequently resulted in performance degradation: reward curves began to oscillate more strongly, and agents often converged to degenerate behaviors such as drifting, repeated collisions, or stalled motion. Consequently, 100 iterations were selected as the standard training duration for subsequent experiments to ensure consistent and stable evaluation.

Several algorithm-level adjustments were explored, including increasing entropy regularization to encourage exploration, reducing the learning rate to improve stability, increasing the critic loss coefficient to enhance value estimation, and modifying the network architecture to increase representational depth. While some of these changes slightly altered training smoothness, none produced meaningful behavioral improvements. Increased entropy led to drifting behavior without convergence, learning rate reduction slowed adaptation without stabilizing coordination, and critic and architectural modifications failed to alter the learned greedy traversal strategy.

Overall, optimization-level modifications alone were insufficient to induce structured parallel coordination or reliable parking behavior. These findings suggest that the primary limitation was not model capacity or update stability, but rather the structure of the reward signal itself. This observation motivated the subsequent exploration of reward shaping strategies, which ultimately led to the emergence of successful coordination behaviors.

4.3.1 Reward Shaping and Task-Level Modifications for Parallel Passing

After optimization-level adjustments proved insufficient, attention shifted to reward shaping and task-level modifications, which led to the first successful emergence of structured parallel passage behavior. The key changes relative to the baseline included setting `collision_reward` to -0.015 , reducing `max_speed_1` to 0.25 , fixing `max_steps` at 400 , enabling `observe_joint_angle`, activating `use_vel_controller`, and reducing the `on_policy_minibatch_size` from 400 to 300 . All remaining parameters were kept identical to the default configuration.

These modifications collectively reshaped the optimization landscape. The stronger collision penalty discouraged aggressive and misaligned movements, while the reduced speed limited abrupt directional changes. Enabling joint-angle observation enriched the state representation with alignment information, improving value estimation under coupled dynamics. Most critically, reducing the minibatch size played a decisive role: with the default minibatch size, the same task parameters failed to produce coordination, whereas a minibatch size of 300 consistently yielded stable parallel traversal. This suggests that update granularity and gradient variance significantly influenced the emergence of fine-grained synchronized behavior.

As shown in Figures 4 and 5, the modified configuration achieved faster reward growth, higher final reward values, and reduced long-term variance compared to baseline. Critic loss (Figure 6) exhibited improved stability, reflecting more consistent value estimation. Qualitative evaluation confirmed that agents executed coordinated parallel passage with reduced bumping; however, terminal parking precision remained suboptimal, motivating further refinement toward improved goal stabilization.

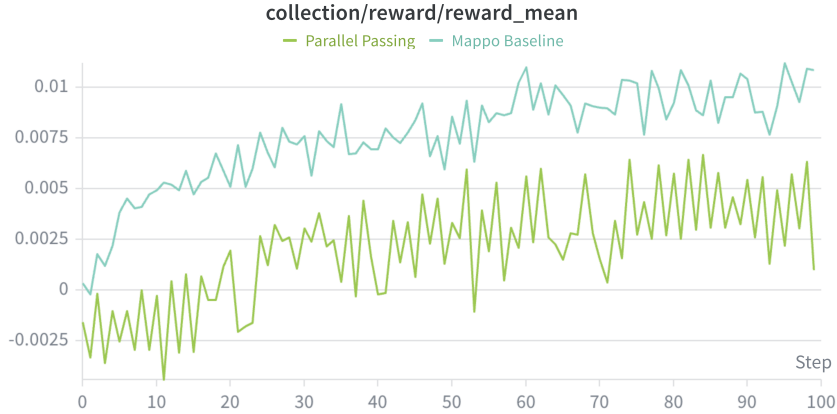


Figure 4: Training mean reward for the parallel passing configuration. The curve shows faster convergence and reduced long-term variance compared to the baseline setup.

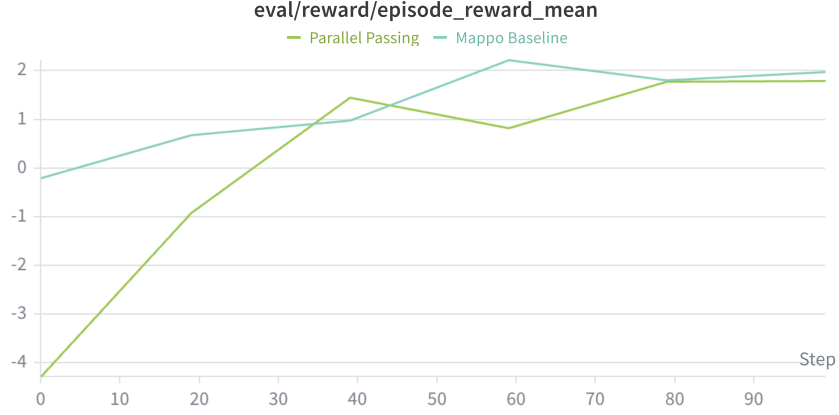


Figure 5: Evaluation episode reward for the parallel passing configuration, confirming stable generalization and improved coordination.

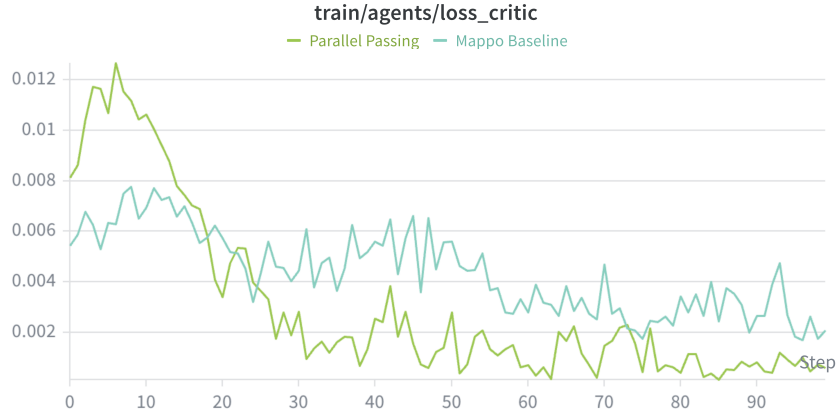


Figure 6: Critic loss during training for the parallel passing configuration, showing improved stability and smoother value estimation compared to baseline.

4.3.2 Parallel Parking Emergence and Behavioral Trade-offs

Following the successful induction of parallel passage behavior, further refinements were conducted to improve terminal parking precision near the goal region. While the previous configuration emphasized controlled traversal through constrained gates, the objective of this phase was to enhance positional stabilization at the final target.

Two distinct configurations emerged as particularly strong candidates for high-quality parking performance.

Run 1: Aggressive Convergence with Near-Perfect Parking. The first configuration prioritized terminal reward accumulation and faster movement dynamics. In this setting, `collision_reward` was set to -0.01 , `max_speed_1` was increased to 0.7 , and `max_steps` remained fixed at 400 . Joint-angle observation was disabled, while the velocity controller remained enabled (`use_vel_controller = true`). These adjustments emphasized rapid goal-directed movement and positional stabilization over strict alignment during passage.

Under this configuration, the agents achieved the most precise parking position across all experiments, settling almost exactly on the target points. However, this success came at the cost of traversal quality. Passage through the environment was aggressive and unstructured, characterized by frequent collisions and greedy selection of the larger gate. Coordination during traversal was weak, though terminal stabilization was strong.

Reward curves indicate rapid accumulation of terminal reward but increased oscillatory critic dynamics (Figure 7). The higher maximum speed enabled faster goal acquisition but reduced the smoothness of cooperative movement.

Run 2: Controlled Greedy Passage with Stable Parking. The second configuration attempted to balance traversal control and terminal stabilization. In this setting, `collision_reward` was set to -0.015 , `fixed_passage` was disabled, and `max_speed_1` was reduced to 0.25 , while `max_steps` remained fixed at 400 . Joint-angle observation was enabled (`observe_joint_angle = true`), and the positional shaping factor was increased to 1.5 to strengthen terminal proximity incentives. The velocity controller remained active (`use_vel_controller = true`), and the `on_policy_minibatch_size` was set to 300 .

This configuration resulted in slightly less precise final parking compared to Run 1, but significantly improved traversal stability. Although agents still selected the larger gate (i.e., greedy passing), the movement was coordinated and controlled, with reduced collision frequency and smoother alignment.

The increased position shaping factor strengthened terminal proximity gradients, improving stabilization at the goal region, while the reduced minibatch size preserved coordinated update dynamics. As shown in Figure 8, the parking-focused configuration achieved higher final rewards than baseline. However, compared to the parallel passing setup, it exhibited greater oscillatory variance during traversal. Critic loss stabilized over time but showed higher initial volatility, indicating increased sensitivity to positional reward shaping in early training stages.

Inability to Combine Parallel Passing and Optimal Parking. Despite extensive experimentation, a configuration achieving both consistently parallel passage through narrow gates and near-perfect terminal parking was not obtained.

Empirically, configurations emphasizing collision penalties, reduced speed, and joint-angle observability favored coordinated parallel traversal but weakened aggressive goal stabilization. Conversely, configurations increasing speed and position shaping im-

proved terminal precision but encouraged greedy gate selection and collision-heavy traversal.

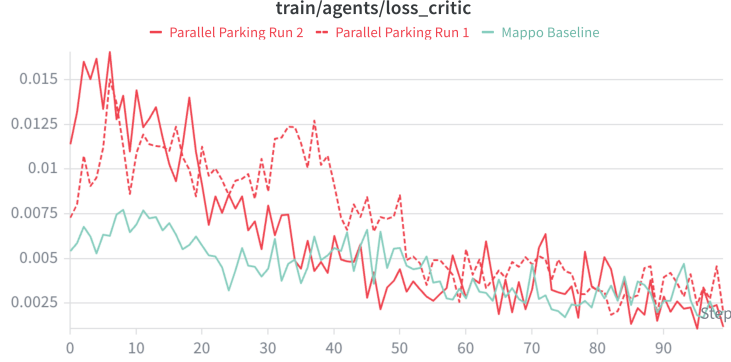


Figure 7: Critic loss during training for the parking-focused configuration (Run 1), showing increased oscillatory dynamics compared to the parallel passing setup.

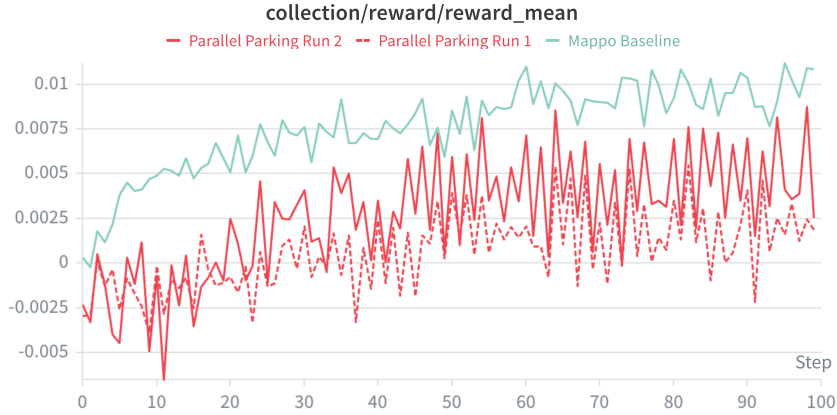


Figure 8: Training mean reward for the parking-focused configuration (Run 2), demonstrating improved final reward values relative to baseline, but with increased variance during traversal phases.

Reward Conflict and Objective Misalignment. The failure to obtain a single configuration that reliably achieved both stable parallel passage and near-perfect parking is consistent with a reward conflict between an *intermediate coordination objective* (gate alignment) and a *terminal proximity objective* (distance-to-goal minimization). In the parallel-passing regime, agents learn fine-grained control to align horizontally with the gate before traversal. This alignment persists after passing the gate, placing

the coupled agents in an orientation that is already compatible with the parking targets; as a result, only small positional corrections are needed near the end state, and the policy can “coast” into the goal region with limited additional reorientation.

In contrast, parking-focused configurations encourage a more direct, goal-oriented traversal that often crosses the gate with a near-vertical orientation relative to both the gate and the parking line. This geometry produces slight overshooting of the optimal parking positions, after which the mechanically coupled agents must execute corrective maneuvers while remaining linked. The leading (typically smaller) agent becomes displaced farther from its target and initiates stronger corrections, which induces misalignment forces on the partner agent on the opposite side of the goal line. Because the reward provides continuous positive feedback primarily for reducing distance to the endpoints (rather than explicitly rewarding correct alignment during the correction phase), these post-gate adjustments are driven by proximity incentives and can destabilize the structured parallel formation learned for passage.

Overall, the scalarized reward formulation induces a multi-objective tension: shaping that strengthens gate-alignment control promotes clean parallel traversal but reduces pressure for aggressive terminal refinement, whereas shaping that emphasizes terminal distance promotes precise parking but encourages traversal geometries that require disruptive post-gate reorientation. Under such competing gradient signals, MAPPO tends to converge to locally stable behavioral regimes that optimize one objective more strongly than the other, rather than a unified policy that simultaneously optimizes both.

4.4 Give Way Task

Task description and coordination requirements. The *Give Way* task is a two-agent coordination problem set in a narrow corridor with two goal regions. Each agent must reach the goal associated with its own color. The initial configuration places agents in front of each other’s goal, meaning they must effectively swap positions.

The corridor contains a narrow asymmetric opening near the middle that can accommodate only one agent at a time. As a result, the optimal strategy requires *heterogeneous behavior*: one agent must temporarily yield by moving into the opening (or waiting area) to allow the other agent to pass, after which roles reverse implicitly as both continue toward their goals. Each agent observes its position, velocity, and the relative position to its goal. The scenario terminates when both agents reach their respective goals.

The main difficulty is breaking symmetry and learning consistent right-of-way behavior purely from reward feedback, without explicit communication.

MAPPO. MAPPO showed limited coordination performance on the simplified Give Way task. Under baseline settings, agents moved toward the corridor but progressed very slowly, often failing to complete the passing maneuver within the episode horizon. Returns were moderate, yet evaluation rollouts revealed hesitation rather than decisive yielding behavior.

Task-level modifications, including increased final rewards, longer episode horizons, and enabling relative position observations, improved numerical returns. However, qualitative behavior remained problematic. In several runs, both agents converged to the same waiting position instead of establishing a clear yielding role, resulting in mutual blockage.

Increasing training duration and frame count did not resolve this issue. Agents frequently stalled before or within the corridor, suggesting that MAPPO struggled to consistently learn asymmetric right-of-way behavior even in the simpler two-agent setting. Overall, while learning remained stable, coordination remained incomplete. This behavior is reflected in the training and evaluation reward curves shown in Figures 9 and 10, where MAPPO converges to moderate returns without fully resolving right-of-way conflicts.

MASAC. MASAC showed partially correct coordination on the Give Way task. In several configurations, one agent moved to the waiting position and allowed the other to pass, indicating that asymmetric right-of-way behavior was learned. However, the passing agent often stopped shortly after crossing the middle instead of consistently reaching its goal.

Returns were stable and relatively higher than those observed with MAPPO, but qualitative behavior remained incomplete. Although MASAC captured the basic yielding pattern, it struggled to maintain goal-directed movement after the interaction phase. Overall, coordination was clearer than with MAPPO, yet still not fully reliable or stable. As illustrated in Figures 9 and 10, MASAC achieves higher returns than MAPPO but does not reach the same final performance as MADDPG.

MADDPG. MADDPG was evaluated across a range of algorithm, experiment, and task configurations. Several runs with near-baseline reward settings resulted in symmetric deadlock behavior. In these cases, both agents moved toward the corridor but stopped either in the middle or inside the waiting area, producing stable but moderate returns (approximately 3–7). Although learning curves appeared smooth, evaluation rollouts showed incomplete coordination and unresolved right-of-way situations.

Adjusting task-level parameters had a stronger impact than modifying algorithm-level settings. Increasing the positional shaping factor and enabling relative position observations improved negotiation behavior, while very small final rewards or weak shaping led to agents remaining stuck. Some configurations produced partial success, where one agent reached its goal but the other remained blocked. Extending training to 300 iterations did not significantly improve performance beyond the best 100–150 iteration configurations, indicating that reward structure was more critical than longer optimization.

The most successful runs combined moderate reward shaping, enabled relative position observations, and completion-based termination. Under these settings, both agents consistently learned clear yielding behavior and completed the task reliably, achieving the highest and most stable returns. This stronger convergence and improved eval-

uation performance can be observed in Figures 9 and 10, where MADDPG clearly outperforms the other algorithms.

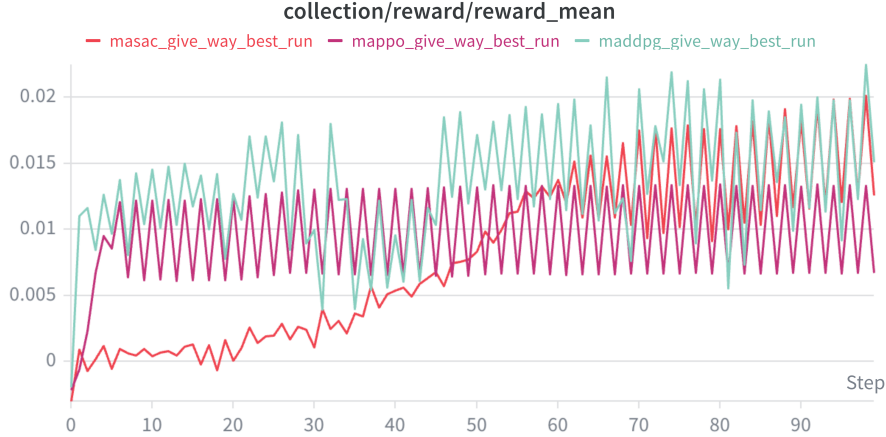


Figure 9: Mean training reward for the Give Way task (best-performing configuration per algorithm).

VDN and QMIX. VDN and QMIX were not further evaluated on the Give Way task. Both methods had already shown weak performance and unstable coordination in the more complex Multi Give Way setting, where agents failed to learn consistent yielding behavior. Given their value-decomposition structure and poor results in the harder scenario, additional experiments on the simplified task were not prioritized. Since MADDPG achieved strong and stable coordination in Give Way, further exploration of VDN and QMIX was unnecessary.

4.5 Multi Give Way Task

Task description and coordination requirements. The *Multi Give Way* task extends the *Give Way* corridor scenario to four agents, each with its own goal region. Agents are positioned such that they must pass through a shared narrow corridor segment to reach their targets, creating repeated multi-agent conflicts in a constrained space.

As in the two-agent version, the corridor geometry prevents simultaneous passing. Agents must therefore negotiate right-of-way through emergent coordination, deciding when to proceed and when to yield to avoid deadlock. This setting increases the coordination complexity due to (i) more frequent encounters, (ii) a larger number of possible yielding patterns, and (iii) stronger risk of mutual blockage. The task remains continuous-control and does not provide explicit communication; coordination must arise from the learned policies and shared incentives.

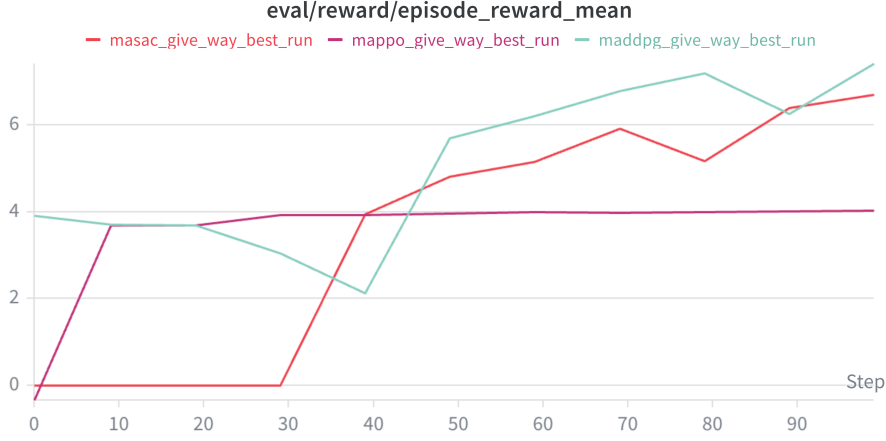


Figure 10: Mean evaluation reward for the Give Way task (best-performing configuration per algorithm).

MAPPO. MAPPO was evaluated using the full baseline configuration, where default algorithm, experiment, and task parameters were left unchanged. Training was generally stable and returns improved over time; however, evaluation rollouts revealed inconsistent coordination. Agents frequently blocked each other in the corridor and failed to reliably establish clear right-of-way behavior.

Further tuning included adjustments to learning rate, entropy coefficient, advantage estimation (λ), minibatch size, number of environments, and episode length. Increasing exploration and extending the horizon improved episode returns in some configurations. The best-performing setup achieved substantially higher rewards, but coordination remained imperfect, with agents often hesitating or partially blocking each other rather than consistently resolving the passage.

Overall, MAPPO demonstrated meaningful learning progress but did not achieve stable and reliable negotiation behavior in the Multi Give Way task. This plateau in performance can be observed in Figures 11 and 12, where MAPPO converges to moderate returns without consistently resolving corridor conflicts.

MASAC. MASAC was evaluated starting from the full baseline configuration, where the default algorithm, experiment, and task parameters provided by BenchMARL were left unchanged. Under this setup, training was stable and returns converged smoothly. However, evaluation rollouts often showed incomplete coordination, with agents moving toward the corridor but failing to consistently resolve the right-of-way situation.

Further tuning included adjustments to algorithm, experiment, and task parameters. Learning rate, entropy coefficient initialization, critic parameter sharing, episode length, reward shaping factors, and collision penalties were modified. These changes improved stability and increased returns in some configurations, but the qualitative

behavior remained similar. Agents frequently hesitated or became partially blocked instead of clearly deciding which one should proceed.

Compared to MADDPG and MAPPO, fewer tuning iterations were performed for MASAC, as the coordination pattern did not significantly change across configurations. Although learning remained stable, MASAC did not consistently achieve decisive right-of-way behavior in the Multi Give Way task. As shown in Figures 11 and 12, MASAC exhibits smooth convergence but remains limited in final coordination quality compared to MADDPG.

MADDPG. Across multiple runs, MADDPG showed mixed results on the Multi Give Way task. With near-baseline settings, agents moved toward the corridor but often got blocked in the middle. Returns were stable (around 3–4), but coordination remained incomplete, with agents hesitating or stopping when facing each other.

Changes at the algorithm level (e.g., critic sharing, collision penalties, exploration annealing) had limited effect on breaking this blockage pattern. Agents typically learned to approach the passage but failed to consistently establish clear right-of-way behavior.

Clear improvements appeared only after task-level modifications. Stronger positional shaping, adjusted friction, and higher final rewards increased returns significantly (around 15) and enabled partial success, with at least two agents reaching their goals. However, overly aggressive penalties or extended 300-iteration training degraded performance, leading to unstable or reversed behaviors. Overall, MADDPG was highly sensitive to reward shaping and task design. Nevertheless, under tuned task configurations it achieved substantially higher returns than the other algorithms, as reflected in Figures 11 and 12, where the performance gap becomes clearly visible in both training and evaluation metrics.

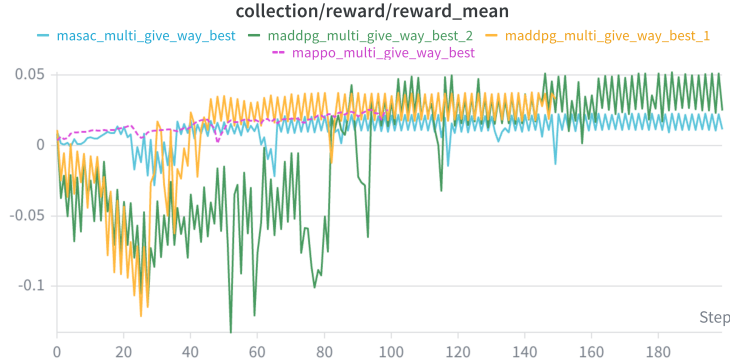


Figure 11: Mean training reward for the Multi Give Way task (best-performing configuration per algorithm).

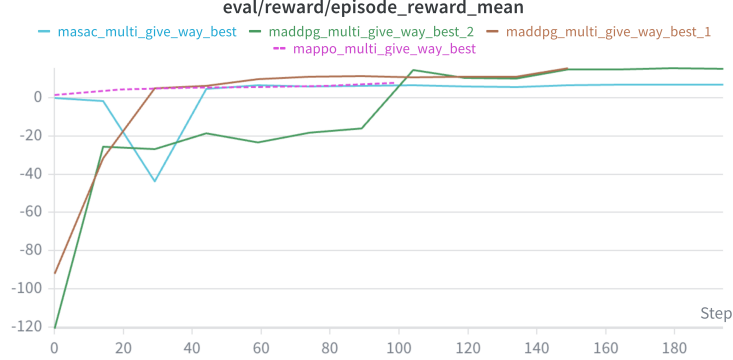


Figure 12: Mean evaluation reward for the Multi Give Way task (best-performing configuration per algorithm).

VDN and QMIX. VDN and QMIX performed poorly on the Multi Give Way task, with negative returns and unstable evaluation behavior. Agents moved in an uncoordinated manner, frequently colliding or blocking each other without learning effective yielding strategies.

These methods rely on value decomposition and are better suited for discrete and more additive cooperative problems. The continuous control and strong inter-agent coupling in this task made them unsuitable, so further tuning was not pursued.

4.6 Reverse Transport Task

Task description and coordination requirements. The *Reverse Transport* task is a cooperative continuous-control scenario in the VMAS suite. A set of n_{agents} agents and a single package (by default $n_{\text{packages}} = 1$) are spawned at random positions in a bounded region, together with a goal location. The objective is to move the package to the goal region. The episode terminates when the package overlaps with the goal.

The reward is shared across all agents and is proportional to the change in distance between the package and the goal: pushing the package closer yields positive reward, while pushing it away yields negative reward. Once the package overlaps with the goal, it is marked as completed and its contribution to the reward becomes zero. Each agent observes its own position and velocity, relative positions to the package, the package velocity, relative positions between the package and the goal, and a completion flag indicating whether the package is on the goal.

A key challenge is the package mass: by default it is significantly heavier than an individual agent, making solo pushing ineffective. Successful policies therefore require synchronized force application and stable joint motion, since uncoordinated contacts can cause oscillations or accidental movement in the wrong direction.

MAPPO. MAPPO achieved successful coordination on the Reverse Transport task already under near-baseline settings. Agents consistently learned to apply synchronized force to the object and transport it toward the goal without instability. Returns were high and stable across evaluation episodes, and the object was reliably delivered within the episode horizon.

Additional runs with slightly modified task parameters (e.g., adjusted package mass and episode length) maintained successful behavior. Overall, MAPPO demonstrated strong stability and effective cooperative transport without requiring extensive tuning. This consistent convergence and high final performance can be observed in Figures 13 and 14, where MAPPO maintains stable and superior reward curves throughout training and evaluation.

MADDPG. MADDPG showed mixed performance depending on task configuration. In one configuration with heavier package mass, agents moved toward the goal but progressed too slowly to complete the task within the episode horizon. Although numerical returns were relatively high due to shaping rewards, the object was not consistently delivered.

After reducing the package mass and maintaining moderate shaping and completion-based termination, MADDPG successfully learned coordinated transport. In these settings, agents synchronized their movement and reliably reached the goal. This indicates that MADDPG was more sensitive to physical task parameters than MAPPO, particularly object mass and time constraints. As illustrated in Figures 13 and 14, the best MADDPG configuration approaches MAPPO performance after tuning, but convergence is less stable across settings.

MASAC. MASAC struggled to achieve stable transport behavior. In several runs, agents moved very slowly toward the goal, leading to incomplete episodes. In other configurations, agents moved in incorrect directions or displayed unstable motion patterns, including drifting away from the target.

Even with increased episode horizons and stronger shaping rewards, coordination remained inconsistent. While some runs showed partial progress, MASAC did not reliably learn stable object transport under the tested configurations. This instability is reflected in Figures 13 and 14, where MASAC exhibits slower convergence and lower final returns compared to MAPPO and MADDPG.

VDN and QMIX. VDN and QMIX were not evaluated on the Reverse Transport task. Given their weak performance in the corridor-based tasks and the continuous control nature of Reverse Transport, further experiments with value decomposition methods were not prioritized.

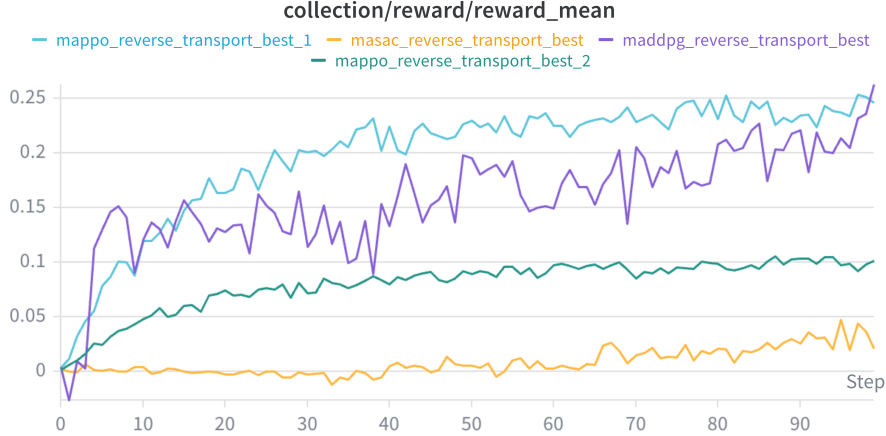


Figure 13: Mean training reward for the Reverse Transport task (best-performing configuration per algorithm).

4.7 Balance Task

Task description and coordination requirements. The *Balance* task requires four agents to cooperatively stabilize a package while simultaneously transporting it toward a target location. The reward combines a positional shaping component, encouraging movement toward the goal, and a balancing component that penalizes excessive tilt or instability. Successful policies must therefore achieve a trade-off between maintaining stable alignment and progressing in the target direction. Poor coordination typically results either in oscillatory behavior or in agents prioritizing stabilization without sufficient forward moving.

MAPPO. MAPPO successfully solved the Balance task under the baseline configuration. Agents rapidly learned synchronized behavior, maintaining stable alignment while steadily moving toward the goal. Training was stable throughout, and evaluation performance remained consistently high (evaluation mean episode reward 127).

MADDPG. MADDPG showed mixed behavior across configurations. Under baseline settings, agents achieved moderate rewards but did not consistently complete the task. After modest task-level adjustments (reduced package mass and shaping refinement), performance improved substantially. The best configuration achieved strong balancing and reliable target reaching (evaluation mean episode reward 121), representing the strongest overall performance among the tested methods. This indicates sensitivity to physical parameters but strong capability once appropriately tuned.

MASAC. MASAC did not consistently achieve full task completion. Under baseline settings, agents displayed partial coordination but frequently deviated from the target

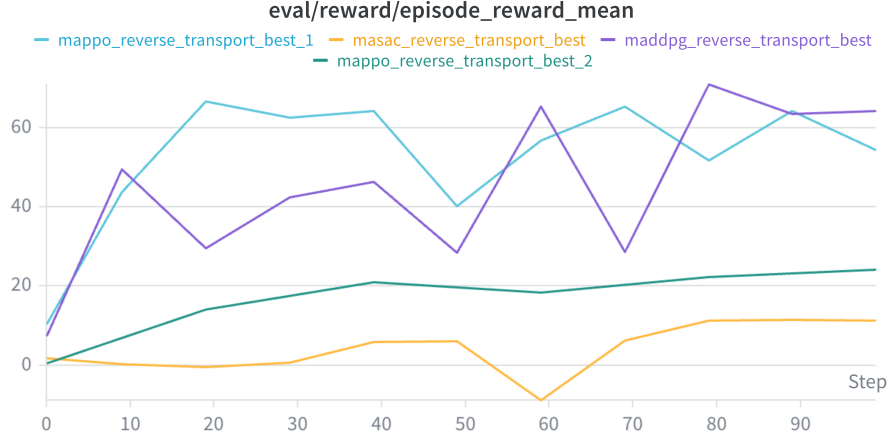


Figure 14: Mean evaluation reward for the Reverse Transport task (best-performing configuration per algorithm).

direction. With modified task parameters, forward motion improved and evaluation rewards increased (up to 81), yet agents often prioritized stabilization over sustained movement toward the goal. Overall, MASAC demonstrated partial balancing capability but weaker directional control compared to MAPPO and tuned MADDPG.

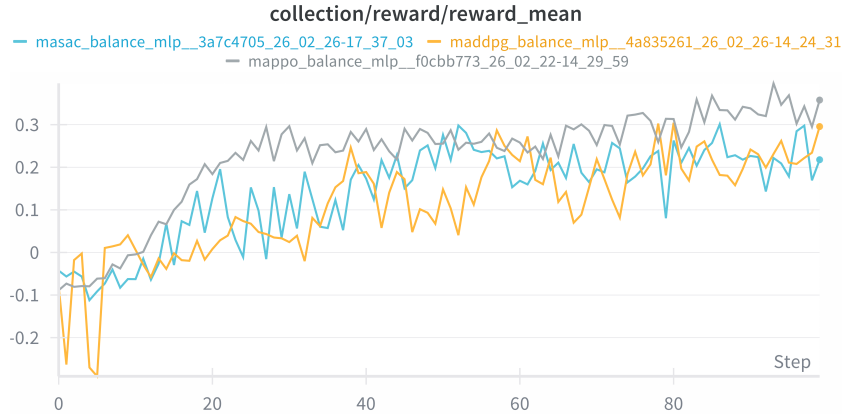


Figure 15: Mean training reward for the Balance task (best-performing configuration per algorithm).

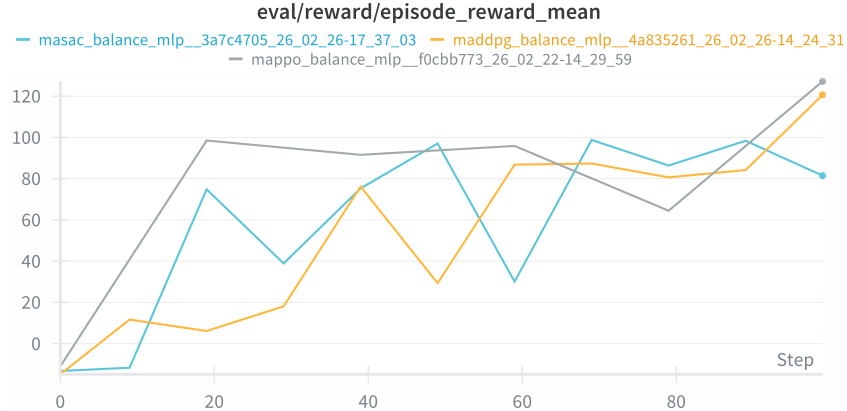


Figure 16: Mean evaluation reward for the Balance task (best-performing configuration per algorithm).

5 Future Work

Although some algorithms showed strong results, further work is needed to better understand why others did not perform as well. In particular, MASAC did not achieve consistent or reliable coordination across the evaluated tasks. While training was often numerically stable, the learned behaviors were incomplete or unstable. Future experiments could include longer training durations, more extensive hyperparameter tuning, or different entropy settings to better evaluate its potential.

Another direction for future work is deeper exploration of model configurations. In this work, the MLP architecture was used consistently, and only limited experiments were conducted with alternative architectures. Since no significant differences were observed in the initial tests, model tuning was not prioritized. However, more systematic exploration of network depth, width, or recurrent architectures could potentially improve performance in tasks that require more complex coordination.

Future studies could also examine scalability by increasing the number of agents or introducing more challenging environment variations. Adding communication mechanisms or testing hybrid training approaches may further improve coordination in complex multi-agent settings.

6 Conclusion

This work presented a systematic benchmarking study of cooperative MARL algorithms in continuous-control VMAS environments using the standardized BenchMARL pipeline. Across Joint Passage Size, Give Way/Multi Give Way, Reverse Transport, and Balance, we found that algorithm performance is strongly task-dependent: MAPPO consistently produced the most stable learning dynamics and the strongest coordina-

tion in constrained passage and balance-style tasks, while MADDPG (and in some cases MASAC) was more effective in interaction-heavy right-of-way negotiation when task rewards and physical parameters were tuned appropriately. A key empirical finding is that reward shaping and task configuration can induce qualitatively different-and sometimes competing-behavioral regimes within the same environment (e.g., parallel passage versus precise terminal parking in Joint Passage Size), emphasizing that reproducible evaluation must combine quantitative learning curves with behavioral inspection. Overall, the results highlight both the utility of unified benchmarking for fair comparison and the central role of task design in determining whether cooperative policies emerge reliably in coupled continuous-control settings.

References

- [1] Matteo Bettini, Ryan Kortvelesy, Jan Blumenkamp, and Amanda Prorok. Vmas: A vectorized multi-agent simulator for collective robot learning. In *International Symposium on Distributed Autonomous Robotic Systems*, pages 42–56. Springer, 2022.
- [2] Matteo Bettini, Amanda Prorok, and Vincent Moens. Benchmarl: Benchmarking multi-agent reinforcement learning. *Journal of Machine Learning Research*, 25(217): 1–10, 2024.
- [3] Lucian Busoniu, Robert Babuska, and Bart De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008. doi: 10.1109/TSMCC.2007.913919.
- [4] Lorenzo Canese, Gian Carlo Cardarilli, Luca Di Nunzio, Rocco Fazzolari, Daniele Giardino, Marco Re, and Sergio Spanò. Multi-agent reinforcement learning: A review of challenges and applications. *Applied Sciences*, 11(11):4948, 2021.
- [5] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Handbook of Reinforcement Learning and Control*, pages 321–384, 2021.